Capstone Design Document + V&V Document

Version 0
Group 13
COMPSCI 4ZP6
Dr. Mehdi Moradi
Jan 23rd, 2026

Abdul-Hadi Siddiqui	sidda1@mcmaster.ca	400386070
Ankush Sarkar	sarkaa16@mcmaster.ca	400376621
Asad Muhammad	muhama23@mcmaster.ca	400368869
Eduardo Salvacion	salvacie@mcmaster.ca	400393061
Jayesh Anil	anilj@mcmaster.ca	400384263
Michael Zhang	zhanz479@mcmaster.ca	400387890
Viransh Shah	shahv47@mcmaster.ca	400394334

Table of Contents

Table of Contents 2
1. Purpose 4
2. Component Diagram 4
3. Requirements 5
3.1 Presentation Layer 5
3.1.1 Priority 0 5
3.1.2 Priority 1 6
3.1.3 Priority 2 6
3.2 Business Layer 7
3.2.1 Priority 0 7
3.2.2 Priority 1 7
3.2.3 Priority 2 7
3.3 Data Layer 7
3.3.1 Priority 0 7
3.3.2 Priority 1 7
3.4 Gemini 8
3.4.1 Priority 0 8
3.5 Google ML Kit + Expo Voice 8
3.5.1 Priority 0 8
3.5.2 Priority 1 8
3.6 State Management 8
3.6.1 Priority 0 8
3.6.2 Priority 1 8
3.7 Database/Remote Infrastructure 9
3.7.1 Priority 0 9
3.7.2 Priority 1 9
4. Components 9
4.1 Presentation Layer 9
4.1.1 Normal Behaviour 9
4.1.2 API and Data Structures 9
4.1.3 Implementation 10
4.1.4 Potential Undesired Behaviors 10
4.2 Business Layer 10
4.2.1 Normal Behaviour 10
4.2.2 API and Data Structures 10
4.2.3 Implementation 10
4.2.4 Potential Undesired Behaviors 10
4.3 Data Layer 11
4.3.1 Normal Behaviour 11

4.3.2 API and Data Structures..... 11
4.3.3 Implementation..... 11
4.4.4 Potential Undesired Behaviors..... 11
4.4 Gemini.....12
4.4.1 Normal Behaviour.....12
4.4.2 API and Data Structures.....12
4.4.3 Implementation.....12
4.4.4 Potential Undesired Behaviors..... 12
4.5 Google ML Kit + Expo Voice..... 12
4.5.1 Normal Behaviour.....12
4.5.2 API and Data Structures.....12
4.5.3 Implementation.....12
4.5.4 Potential Undesired Behaviors.....13
4.6 State Management.....13
4.6.1 Normal Behaviour.....13
4.6.2 API and Data Structures.....13
4.6.3 Implementation.....13
4.6.4 Potential Undesired Behaviors.....13
4.7 Database/Remote Infrastructure..... 13
4.7.1 Normal Behaviour.....13
4.7.2 API and Data Structures.....14
4.7.3 Implementation..... 14
4.7.4 Potential Undesired Behaviors.....14
5. UI Details..... 15
5.1 General..... 15
5.2 Color.....15
5.3 Font..... 15
5.4 Placement.....15
5.5 Figma.....15
6. V&V Plan..... 15
6.1 Presentation Layer..... 15
6.1.1 Unit Tests.....15
6.1.2 Performance Tests and Metrics.....16
6.1.3 Snapshot Tests..... 16
6.2 Business Layer.....16
6.2.1 Unit Tests.....16
6.2.2 Performance Tests and Metrics.....16
6.3 Data Layer.....17
6.3.1 Unit Tests..... 17
6.3.2 Performance Tests and Metrics.....17
6.4 Gemini.....17

6.4.1 Unit Tests..... 17
6.4.2 Performance Tests and Metrics.....17
6.5 Google ML Kit + Expo Voice.....17
6.5.1 Unit Tests..... 17
6.5.2 Performance Tests and Metrics.....18
6.6 State Management.....18
6.7 Database/Remote Infrastructure.....18
6.7.1 Unit Tests..... 18
6.7.2 Performance Tests and Metrics.....18

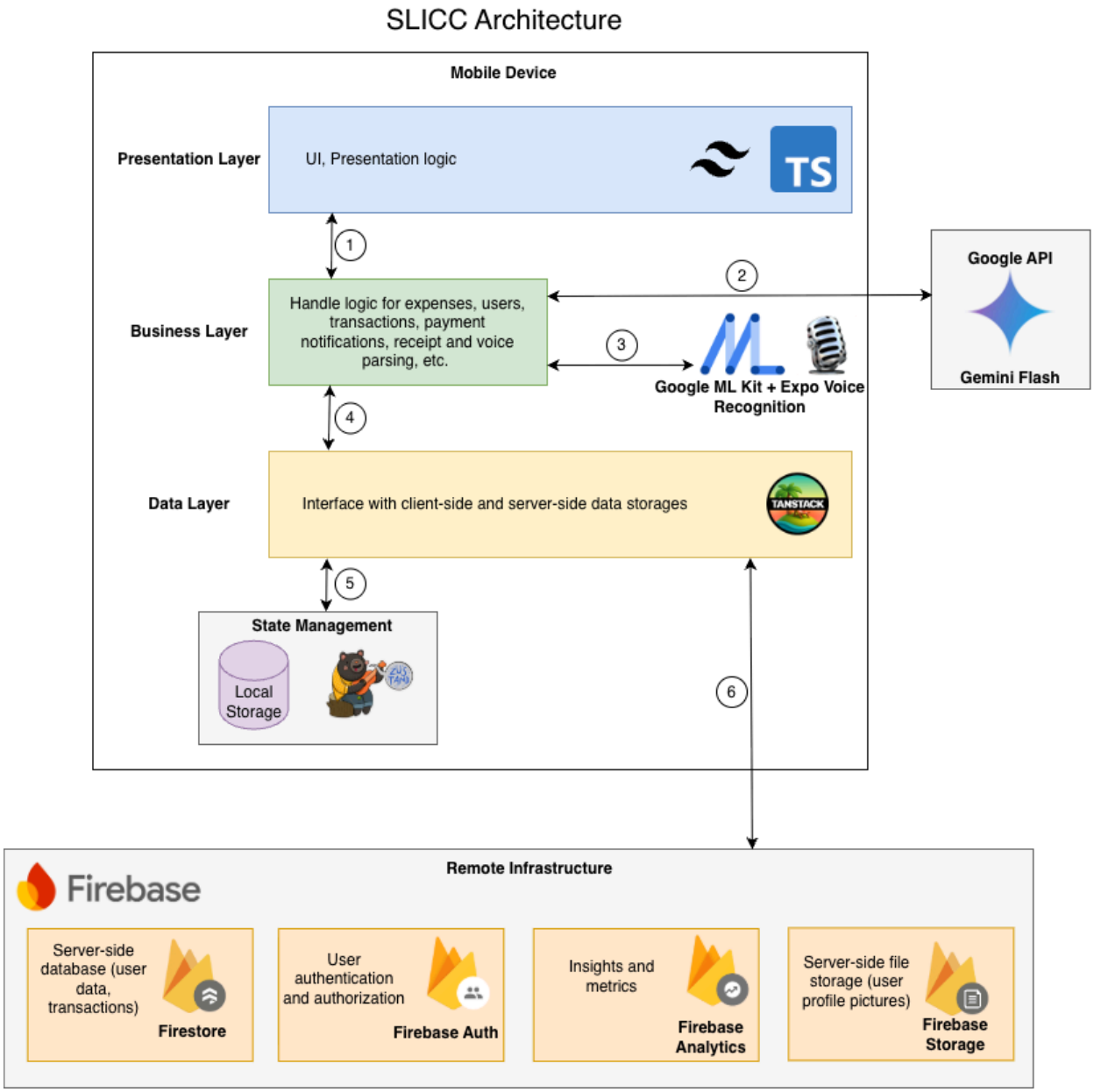
1. Purpose

The Slicc project simplifies group expense management by combining event planning, expense splitting through OCR and voice recognition, and payment coordination in a single application. Key features include automatic receipt scanning, voice-based bill splitting, payment tracking by expense creators, configurable tax/tip handling, group-based event organization, and flexible payment integration for both app users and non-users.

This design document outlines the technical architecture, component design, and validation/verification plan for the Slicc system. It explains the relationship between components and how the system meets the requirements defined in Software Requirements Specification (SRS).

2. Component Diagram

Components:
Presentation Layer
Business Layer
Data Layer (Move React Query here)
Gemini
Google ML Kit + Expo Voice
State Management (Combine Zustand + Local data)
Database/Remote Infrastructure



3. Requirements

3.1 Presentation Layer

3.1.1 Priority 0

- The host user should be able to manually add a list of expenses
- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses
- The host user should be able to manually split the expenses among other people or users

- The host user should be able to vocally split the expenses among other payee users through the app (voice detection)
- User authentication
    - The user should be able to sign-up and login with their email, or associated Google/Apple account
    - The user should be able to link their e-transfer email or paypal/venmo details to seamlessly make payments when needed
    - The user should be able to access their recent expenses and expense history

3.1.2 Priority 1

- The user can create events and add other users
    - Users will be added through their unique identifier
    - The user can also create stand-in users for people who do not have the app
        - The stand-in can have either a phone number or email for the purpose of sending a payment reminder
        - The user can also give the stand-in a location for recommendation purposes
    - Users of the app should be able to add dates and times they are available
        - Availability for stand-in users can be changed by other app users that are in the group
    - The user can see a list of other event members
    - The user can set a location for the event
- The user can search through nearby locations
    - Locations can be filtered and sorted by based on an input location
    - Locations can be filtered and sorted by based on average of event members location
    - Locations can be filtered and sorted by based on dietary restrictions for

3.1.3 Priority 2

- The user can create events and add other users
    - The user can export the event date and time to their Google/Apple calendar
- The user can access an in-app map that displays recommended locations of interest (restaurants, karaoke, etc.)
- Event members should be able to access a shared noteboard
    - The user can access the noteboard quickly from anywhere in the event interface

3.2 Business Layer

3.2.1 Priority 0

- The host user should be able to manually add a list of expenses
- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses
- The host user should be able to manually split the expenses among other people or users
- The host user should be able to vocally split the expenses among other payee users through the app (voice detection)
- The payee user should be redirected to their preferred payment method when a payment is requested
    - For people who need to make payments but do not have the app, the host user can input their email or phone number to send an payment notification

3.2.2 Priority 1

- The user can search through nearby locations
    - Locations can be filtered and sorted by based on an input location
    - Locations can be filtered and sorted by based on average of event members location
    - Locations can be filtered and sorted by based on dietary restrictions for

3.2.3 Priority 2

- The user can create events and add other users
    - The user can export the event date and time to their Google/Apple calendar
    - The user can send and receive messages from other event members

3.3 Data Layer

3.3.1 Priority 0

- User authentication
    - The user should be able to sign-up and login with their email, or associated Google/Apple account
    - The user should be able to access their recent expenses and expense history

3.3.2 Priority 1

- The user can create events and add other users
    - Users will be added through their unique identifier
    - The user can also create stand-in users for people who do not have the app

        - The stand-in can have either a phone number or email for the purpose of sending a payment reminder
        - The user can also give the stand-in a location for recommendation purposes
    - Users of the app should be able to add dates and times they are available
    - The user can see a list of other event members
    - The user can set a location for the event

3.4 Gemini

3.4.1 Priority 0

- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses

3.5 Google ML Kit + Expo Voice

3.5.1 Priority 0

- The host user should be able to scan a receipt and be given the parsed and itemized list of expenses
- The host user should be able to vocally split the expenses among other payee users through the app (voice detection)

3.5.2 Priority 1

- The app can be used offline
    - All expense splitting functionality should be available and work offline
        - The user can manually input a list of expenses
        - The user can scan a receipt and get a parsed list of expenses
        - The user can manually split the list of expenses
        - The user can split the bill through vocal recognition
    - Offline expense history will be merged when the user comes back online
        - Only if some user has been logged in prior

3.6 State Management

3.6.1 Priority 0

- User authentication
    - The user should be able to access their recent expenses and expense history

3.6.2 Priority 1

- The app can be used offline
    - All expense splitting functionality should be available and work offline

        - The user can manually input a list of expenses
        - The user can scan a receipt and get a parsed list of expenses
        - The user can manually split the list of expenses
        - The user can split the bill through vocal recognition
    - Offline expense history will be merged when the user comes back online
        - Only if some user has been logged in prior

3.7 Database/Remote Infrastructure

3.7.1 Priority 0

- User authentication
    - The user should be able to sign-up and login with their email, or associated Google/Apple account
    - The user should be able to access their recent expenses and expense history

3.7.2 Priority 1

- The user can create events and add other users
    - Users will be added through their unique identifier
    - The user can also create stand-in users for people who do not have the app
        - The stand-in can have either a phone number or email for the purpose of sending a payment reminder
        - The user can also give the stand-in a location for recommendation purposes
    - Users of the app should be able to add dates and times they are available
        - Availability for stand-in users can be changed by other app users that are in the group
    - The user can see a list of other event members
    - The user can set a location for the event

4. Components

4.1 Presentation Layer

4.1.1 Normal Behaviour

- This Layer is responsible for all the front-end that the user sees and interacts with on the app.

4.1.2 API and Data Structures

- User interactions such as button presses, voice input, camera input to operate the app. Data is structured as React components, eventually mapping to either Android or iOS native UI.

4.1.3 Implementation

- Stack: React Native/Expo (SDK 52), TypeScript, NativeWind, Expo Router, React Hook Form + Zod.
- Files: Root layout (src/app/_layout.tsx), dashboard (src/app/(app)/index.tsx), expense screens (src/app/expense/*), camera (src/app/reciept-camera/index.tsx), components (src/components/*).
- Routing: Root Layout with providers -> Authenticated Routes (tabs: home, groups, notifications, settings) -> Expense/Camera/Auth screens. Zustand integration enables automatic re-rendering

4.1.4 Potential Undesired Behaviors

- Cross-Platform Inconsistencies: Mitigation: Platform-specific testing and styling.
- Keyboard Overlap: Mitigation: KeyboardAvoidingView, react-native-keyboard-controller.
- State Loss: Mitigation: Expo Router persistence, draft storage.

4.2 Business Layer

4.2.1 Normal Behaviour

- This layer is responsible for handling logic for expenses, users and transactions for each user. This also handles payment notifications, receipt scanning and voice parsing for users to add the transactions on their receipt and from user input.

4.2.2 API and Data Structures

- Calculate item amount split between users during the creation of expense.
- Parsing receipt scanner API output.
- Parsing voice recognition API output.
- Data structure used will be JSON

4.2.3 Implementation

- Stack: TypeScript, Axios, Zod validation, custom error classes.
- Files: src/api/expenses/, src/api/items/, src/api/people/, src/api/camera-receipt/extract-receipt-info.tsx, src/lib/utils.ts.
- Receipt Flow: Preprocess -> Gemini API -> validate -> confidence scores -> flag low confidence.
- Splitting: Subtotal -> items per person -> distribute tax -> distribute tip -> totals
- Voice: Speech-to-text -> parse intent -> extract entities -> fuzzy match -> execute -> confirm.

4.2.4 Potential Undesired Behaviors

- Voice Errors: Mitigation: Show transcript, undo, manual alternatives.

4.3 Data Layer

4.3.1 Normal Behaviour

- The Data Layer manages data retrieval, caching, persistence, and synchronization between the client and Cloud Firestore.

4.3.2 API and Data Structures

- The Data Layer exposes queries and mutations to the firestore and validates the JSON results with predefined ZOD schemas
    - Expense
        - create expense, delete expense, update user payment status, get expense by id, get event expenses, add user to expense
    - User
        - get user by id, get user by username, create user, set display name, update display name
    - Item
        - create item in expense, update item details, delete items, assign item to user
    - Groups
        - create group, update group details, add member to group, remove member from group
    - Events
        - create event, update event details, send event invite to user, add expense to event

4.3.3 Implementation

- Stack: TanStack Query v5, Firebase Firestore, MMKV, custom sync engine.
- Files: src/api/common/api-provider.tsx, src/api/common/client.tsx, src/api/expenses/, src/api/items/, src/api/users/, src/api/groups/ src/api/events/.
- Offline Sync: Queue tracks operations -> addToQueue persists -> processQueue executes sequentially -> retry failed operations.

4.4.4 Potential Undesired Behaviors

- Sync Conflicts: Mitigation: Last-write-wins.
- Stale Data: Mitigation: StaleTime tuning, pull-to-refresh, real-time listeners.
- Memory Leaks: Mitigation: Cache limits, garbage collection.

4.4 Gemini

4.4.1 Normal Behaviour

- Google Gemini is used for text recognition from receipts scanned by the users. Only the items and their prices are recorded from the scanned receipts. The items and price for each item is coupled together and itemized based on what the API outputs after scanning the receipt.

4.4.2 API and Data Structures

- The Gemini API accepts a preprocessed, base64-encoded receipt image as input and returns structured JSON containing extracted item names, prices, and confidence scores.

4.4.3 Implementation

- Stack: Gemini 2.5 Flash, expo-camera, expo-image-manipulator.
- File: src/api/camera-receipt/extract-receipt-info.tsx.
- Flow: Resize to 1024px -> compress 80% -> JPEG -> base64 -> POST to API -> parse JSON -> validate -> return with confidence scores.

4.4.4 Potential Undesired Behaviors

- API Latency: Mitigation: Progress indicators, timeout/retry logic, offline ML Kit fallback.
- API Cost: Mitigation: Usage limits, result caching, image optimization, cost monitoring.

4.5 Google ML Kit + Expo Voice

4.5.1 Normal Behaviour

- Expo is used for the voice to text pipeline where user speech is transcribed and itemized based on what the user says on the microphone. The items and price mentioned by the user are coupled together and itemized.

4.5.2 API and Data Structures

- Expo voice recognition returns an object with the voice and the metadata which includes the transcript of microphone input as an array of Javascript objects.
- Input is parsed using a parts of speech tagger.

4.5.3 Implementation

- Stack: Firebase ML Kit, expo-speech/react-native-voice.
- Files: ML Kit in src/api/camera-receipt/extract-receipt-info.tsx, voice in Business Layer.

- Voice Flow: Set listeners -> start recognition (en-US) -> return promise with transcript and confidence -> error handling

4.5.4 Potential Undesired Behaviors

- Offline Accuracy: Mitigation: Clear expectations via UI, show confidence scores, fallback to manual
- Noisy Environments: Mitigation: Noise cancellation, show confidence scores, manual correction, fallback to manual.

4.6 State Management

4.6.1 Normal Behaviour

- The State Management component is responsible for maintaining client-side application state that must persist across screens and sessions. This includes authentication state, user preferences etc.

4.6.2 API and Data Structures

- User settings, work in progress expenses, and user authentication are stored via JSON objects for client-side persistence.

4.6.3 Implementation

- Stack: Zustand, react-native-mmkv.
- Files: src/lib/store.ts, src/lib/storage.tsx, src/lib/auth/index.tsx.
- Offline Queue: NetInfo monitors connectivity -> auto-update offline status -> process queue on restore -> sequential execution -> retry failed operations.

4.6.4 Potential Undesired Behaviors

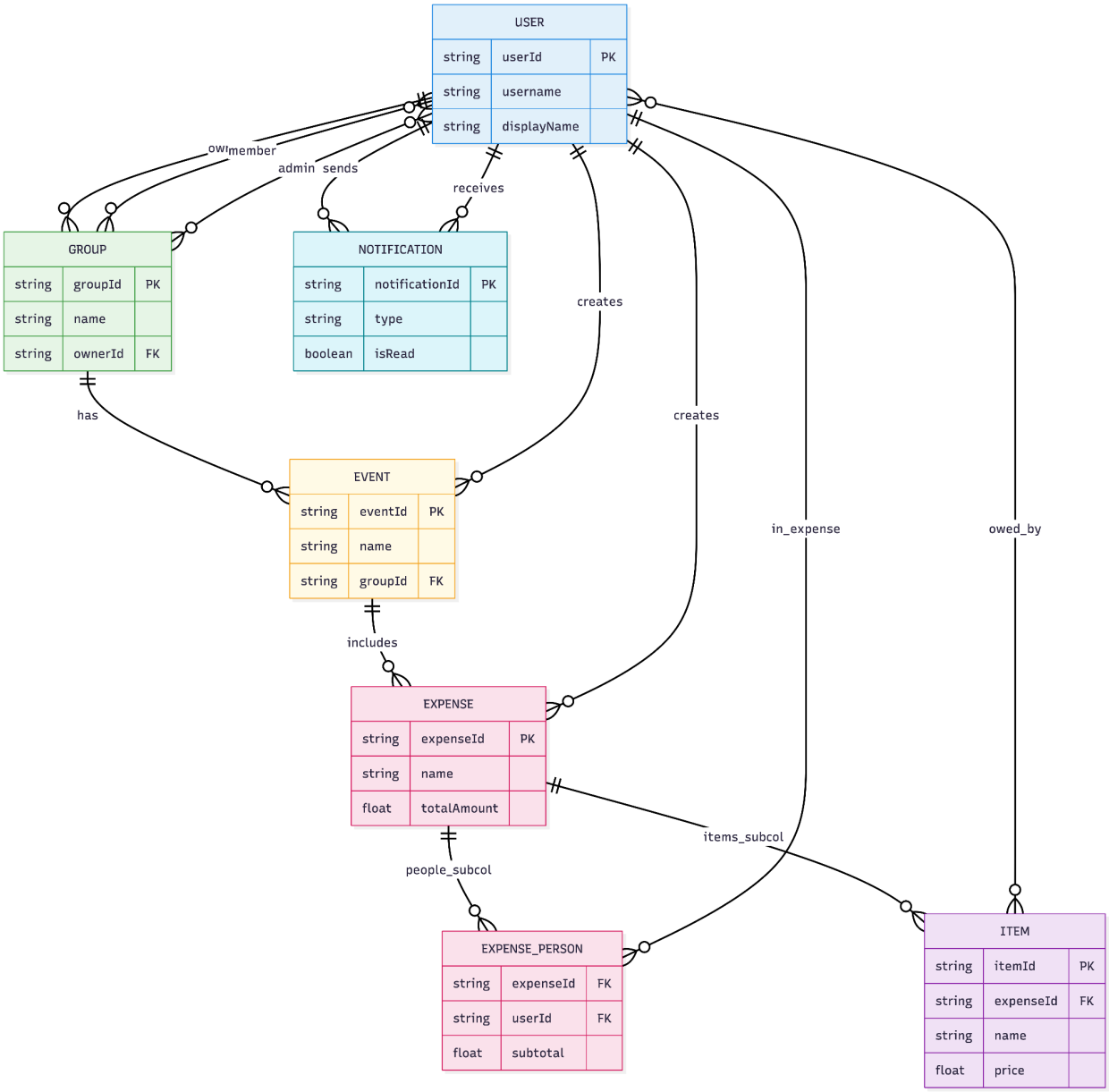
- State Bloat: Mitigation: Keep minimal global state, use React Query for server state, implement slicing.
- Race Conditions: Mitigation: Atomic updates, optimistic locking, avoid derived state.

4.7 Database/Remote Infrastructure

4.7.1 Normal Behaviour

- The Database and Remote Infrastructure component is responsible for securely storing and retrieving persistent application data and handling user authentication.

4.7.2 API and Data Structures



4.7.3 Implementation

- Stack: Cloud Firestore, Firebase Auth, Firebase Storage, Firebase Analytics, @react-native-firebase SDK.
- Config: google-services.json (Android), GoogleService-Info.plist (iOS), firestore.rules, storage.rules.

4.7.4 Potential Undesired Behaviors

- Costs: Mitigation: Aggressive caching, batch writes, optimized queries, cost monitoring.

5. UI Details

5.1 General

The general flow of our application is inspired by many existing apps such as Wealthsimple and Whatsapp. We plan on having 5 tabs at the bottom of the screen to switch between the “main screens”, which are: Home/Dashboard, Expenses, Social, Explore, Settings. Our plan is to have the app feel clean and without too much clutter.

5.2 Color

We are mainly using a black and white color palette. We use greys as the background for dark mode and white at the background for light mode. We also use a blue for an accent for small details and buttons, but this may be adjusted during development. We plan on sticking to a fairly limited color palette to keep our app minimal and sleek.

5.3 Font

The app uses the Inter font for almost all the text. This choice was made because it is sans serif, to maintain the minimal and clean style of the app. We use a standardized font size for normal text and headers. We also decided to only use different fonts for emphasis or stylization of non-functional text.

5.4 Placement

The placement of items stays semi-consistent in the app. We try to maintain a bottom section that holds progressions buttons and showcases “summary” info. The top contains a header and buttons that allow the user to go back if needed.

5.5 Figma

The current/rough designs can be found in the Figma below.
Link: Figma

6. V&V Plan

6.1 Presentation Layer

6.1.1 Unit Tests

Unit testing for the presentation layer focuses on verifying correct UI behaviour, user interactions, and rendering logic across the application. This includes testing React

components such as screens, forms, and lists used for creating user groups, events, and expenses. Scenarios such as creating an expense, adding multiple items, assigning users to split each item, and entering item prices are tested to ensure UI consistency and correctness. React Testing Library is used to simulate user interactions and verify expected behaviour, while manual test scripts are applied for end-to-end UI flows that involve multiple screens.

6.1.2 Performance Tests and Metrics

The primary metric used is initial screen render time, measured from screen load to user interactivity. Performance is considered acceptable if the initial render time is less than 2 seconds on any supported device, including lower-end mobile devices. This threshold ensures a responsive and user-friendly experience and helps prevent perceived lag during navigation or first-time screen loads.

6.1.3 Snapshot Tests

Snapshot tests will be performed on the presentation layer to verify the appearance of the app. Jest (jest-expo) provides a framework to validate screen appearance against a correct developer-reviewed screenshot, while also simulating user actions on those screens. The snapshot similarity threshold will be 90%, ensuring that the user interface conforms to a specified appearance.

6.2 Business Layer

6.2.1 Unit Tests

Unit testing for the business layer focuses on verifying the correctness of core application logic, including balance calculations, expense splitting rules, transaction creation, and notification handling. Example test cases verify that custom split ratios and values produce correct per-user balances and that transactions are created and validated correctly before being persisted. Notification logic is also tested to ensure that users receive appropriate alerts. Test scripts are used to simulate large groups and edge cases, such as maximum participant counts, to validate correctness under load and prevent calculation errors in boundary conditions.

6.2.2 Performance Tests and Metrics

The acceptance criterion requires that expense calculations and related updates render within 200 milliseconds, with only minimal and unnoticeable delay from the user’s perspective. Meeting this threshold ensures that financial computations do not block UI interactions and that the application remains responsive even as the complexity of expense data increases.

6.3 Data Layer

6.3.1 Unit Tests

Unit testing for the data layer focuses on validating correct data fetching, caching, and synchronization between the client and remote services. Tests cover API query and mutation logic, ensuring that data is retrieved, stored, and updated correctly within the application. Unit tests also verify that successful mutations properly invalidate and refetch dependent queries, preventing stale data from being displayed. Mocked API responses are used to simulate both success and failure cases, allowing deterministic and repeatable testing of data-layer behavior.

6.3.2 Performance Tests and Metrics

The primary performance metric is average API response time, measured across multiple requests under normal operating conditions. The acceptance criterion requires that the average API response time remains below 500 milliseconds. Meeting this threshold ensures timely data availability, supports smooth UI interactions, and validates that the data layer meets the performance expectations defined during system design.

6.4 Gemini

6.4.1 Unit Tests

Unit testing for the Gemini Flash component focuses on validating correct request formatting and response parsing when processing receipt data. These tests ensure that receipt images or extracted text are correctly sent to the Gemini API and that responses are parsed into a structured format compatible with the application's expense model. Mocked Gemini API responses are used to simulate both successful and failure scenarios, ensuring robustness without reliance on live API calls.

6.4.2 Performance Tests and Metrics

The acceptance criterion requires that receipt parsing completes in under 5 seconds on average for standard receipts under normal network conditions. Meeting this threshold ensures that users do not experience excessive delays when uploading receipts and validates that the receipt parsing feature performs within acceptable usability limits.

6.5 Google ML Kit + Expo Voice

6.5.1 Unit Tests

Unit testing for the Google ML Kit and Expo Voice Recognition component focuses on validating correct handling of voice-to-text output within the application. The scope includes verifying that the system correctly processes voice input events and

integrates the resulting text into expense-related workflows. Example test cases verify that spoken expense inputs (item names and prices) are correctly converted into structured data required to render expense creation screens. Tests also cover error scenarios, including unclear speech or partial recognition ensuring that the system provides appropriate feedback to the user.

6.5.2 Performance Tests and Metrics

The acceptance criterion requires that voice-to-text processing completes in under 3 seconds under normal operating conditions. Meeting this threshold ensures that voice input is transformed into usable data without noticeable delay, supporting efficient expense entry and maintaining overall application usability.

6.6 State Management

The state management component will not be included for testing as the behavior can be verified when testing the behavior of the data layer.

6.7 Database/Remote Infrastructure

6.7.1 Unit Tests

Unit testing for the database and remote infrastructure focuses on verifying authentication and authorization rules, data validation, and file storage operations. These tests ensure that access control policies are correctly enforced and that only authorized users can read or modify protected resources. The scope includes validating user authentication flows, permission checks, and data integrity constraints across database operations. Example test cases verify that users can only access their own transactions and are prevented from viewing or modifying data belonging to other users. The Firebase Emulator Suite is used to simulate authentication, database, and storage services in a controlled environment, enabling safe and repeatable testing without impacting production data.

6.7.2 Performance Tests and Metrics

The acceptance criteria require that database operations complete in under 500 milliseconds on average, while authentication responses must be returned within 1 second. Meeting these thresholds ensures fast access to user data, smooth session handling, and reliable performance under typical usage conditions, validating that the backend infrastructure meets system performance requirements.